



수학적 모델과 논리

컴퓨터공학부

천세진

목차

2.1 수학적 모델

2.2 논리적 추론

2.3 증명 기술과 프로그램 검증

2.4 응용 : 지식 베이스 시스템

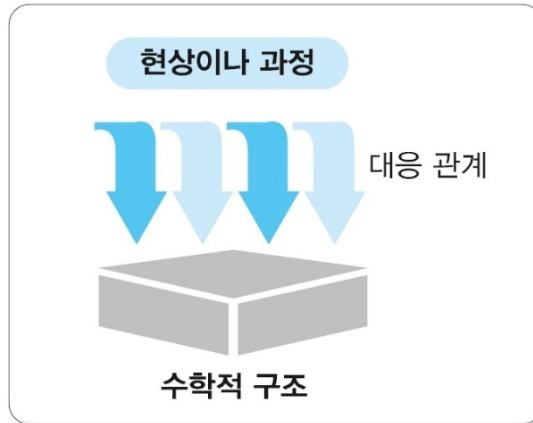
이 장에서 배우는 내용은?

- ❖ 자연과학(물리학, 생물학, 지구과학, 기상학 등), 공학(컴퓨터 과학, 인공지능 등), 사회과학(경제학, 사회학, 정치학 등)에서 많이 사용되는 수학적 모델과 논리적 추론, 수학적 귀납법과 프로그램의 검증
- ❖ 데이터베이스와 인공지능의 응용인 지식 베이스 시스템

2.1 수학적 모델

수학적 모델의 개념

수학적 모델 : 자연 현상의 문제를 수학적 식을 이용하여 푸는 것



[그림 2-1] 수학적 모델의 구성

- 우리가 관심을 가지는 **현상(phenomenon)** 혹은 **과정(process)**에 대해서 **오랜 관찰(observation)**을 통해 그러한 현상 또는 과정이 지닌 특성
- 수학적 모델은 수학적 모델로 되기 위한 **현상이나 과정**이 있어야 하고, 수학적 모델로 되기 위한 **객체(objects)들의 중요한 성질을 표현**하기 위한 **수학적 구조와 현상 또는 과정과 수학적 구조 사이를 대응**시키기 위한 **대응 관계**가 필요

수학적 모델들은 보통 관계와 **변수**로 이루어진다. 관계는 **연산자**에 의해 기술된다. 변수는 시스템 **매개변수**를 추상화한 것으로 양자화된다. 일부 분류 기준은 자신의 구조에 따라 수학적 모델을 위해 사용될 수 있다:

- 선형, 비선형
- 정적, 동적
- 명시적, 암묵적
- 비연속적, 연속적
- 결정적, 확률 과정
- 연역적, 귀납적, 유동적

수학적 모델을 만드는 데 필요한 것

- 정의(definition), 문장(statement), 명제(proposition), 공리(axiom), 논리 연산자, 합성 명제, 논리적 추론(logical inference), 결론, 정리, 증명

2.2 논리적 추론 (Logic Inference)

Definition, Statement, Proposition

정의(definition)

- 용어의 뜻을 명확히 하기 위해서 사용되는 것
- 무정의 정의 : 정의하지 않고 사용할 수 있는 일반적인 용어

문장(statement)

- 무정의 용어와 정의된 용어를 사용하여 만들

정의 2-1 명제

명제^{proposition}란 참이나 거짓 중 단 하나만 갖는 문장을 말하며, 대문자 P, Q, R, ...으로 표시한다. 명제 중에서 명제의 내용이 참이 되는 명제를 참 명제라 하고, 거짓이 되는 명제를 거짓 명제라 한다.

Propositions



진리값(truth value)

- 참과 거짓을 명제의 진리값이라 한다.
- 각각 T(또는 1), F(또는 0)로 표시



명제 변수(propositional variable)

- 진리값이 아직 밝혀지지 않은 임의의 명제
- P, Q, R, ... 등으로 표시

정의 2-2 공리

공리^{axiom}란 참이라고 가정한 최초의 명제이다.



합성 명제 : 주어진 명제들을 합성하여 새로운 명제를 만드는 것

Propositions

- A proposition is a declarative sentence that is either true or false.
- Examples of propositions:
 - The Moon is made of green cheese.
 - Trenton is the capital of New Jersey.
 - Toronto is the capital of Canada.
 - $1 + 0 = 1$
 - $0 + 0 = 2$
- Examples that are not propositions.
 - Sit down!
 - What time is it?
 - $x + 1 = 2$
 - $x + y = z$

Logical Operation (논리 연산)

논리 연산자(and, or, not, exclusive or, imply, equal)

- 논리곱[conjunction, AND, $\wedge$]
- 논리합[disjunction, OR, $\vee$]
- 부정[negation, NOT, $\neg$]
- 배타적 논리합[exclusive or, XOR, $\oplus$]
- 논리 함축[implication, $\rightarrow$]  p이면 q이다, $p \rightarrow q$
- 전제[premise]  참이라고 가정
- 결론[conclusion]
- 역[converse]
- 대우[contraposition]  어떤 조건 명제의 대우(對偶, 영어: contrapositive)는 그 조건 명제의 가정과 결론을 뒤바꾼 뒤 각각 부정을 취하여 얻는 명제
- 이[inverse]
- 동치[**equivalence**, $\Leftrightarrow$]

Propositional Logic

Constructing Propositions

- Propositional Variables: $p, q, r, s, \dots$
- The proposition that is always true is denoted by T and the proposition that is always false is denoted by F .
- Compound Propositions; constructed from logical connectives and other propositions
 - Negation $\neg$
 - Conjunction $\wedge$
 - Disjunction $\vee$
 - Implication $\rightarrow$
 - Biconditional $\leftrightarrow$

Logical Operation (논리 연산)

진리표(truth table)

- 논리 연산에 대한 연산표
- 참과 거짓을 나타내는 변수인 0과 1의 모든 가능한 조합과 이들 각각에 대한 관련 함수의 값을 0과 1(거짓과 참)로 나타내는 진리 함수의 표

논리곱(conjunction), 논리합(disjunction), 부정(negation), Exclusive OR

- 임의의 명제 P와 Q에 대해서

[표 2-1] 논리곱을 나타내는 진리표

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

[표 2-2] 논리합을 나타내는 진리표

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

[표 2-3] 부정을 나타내는 진리표

P	$\neg P$
T	F
F	T

[표 2-4] 배타적 논리합을 나타내는 진리표

P	Q	$P \oplus Q$
T	T	F
T	F	T
F	T	T
F	F	F

합성 명제 $\neg(P \wedge Q) \oplus (\neg P \vee Q)$ 의 진리표를 구하라.

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$	$\neg P$	$\neg P \vee Q$	$\neg(P \wedge Q) \oplus (\neg P \vee Q)$
T	T	T	F	F	T	T
T	F	F	T	F	F	T
F	T	F	T	T	T	F
F	F	F	T	T	T	F

Logical Implication (논리 함축)

논리 함축 (Implication)

[표 2-5] 논리 함축을 나타내는 진리표

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

- 임의의 명제 P와 Q에 대해서 논리 함축 $P \rightarrow Q$ 의 진리값은
P가 참이고 Q가 거짓일 경우에만 거짓이 되고, 그 이외의 경우에는 모두 참이 된다.
- 함축(Implication) / 조건명제(Conditional Proposition) $p \rightarrow q$
 - ✓ "비가 오면 우산을 가지고 간다."는 "비가 온다"라는 조건과 "우산을 가지고 간다"라는 결론이 된다.
 - ✓ 이와 같이 조건과 결론의 관계로 결합된 형태를 함축, 조건명제라고 한다.
 - ✓ **p implies q : p는 q를 함축한다.**
 - ✓ if p then q 또는 p only if q : p면 q다.

합성 명제 $\neg(P \wedge Q) \rightarrow (\neg P \vee Q)$ 의 진리표를 구하라.

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$	$\neg P$	$\neg P \vee Q$	$\neg(P \wedge Q) \rightarrow (\neg P \vee Q)$
T	T	T	F	F	T	T
T	F	F	T	F	F	F
F	T	F	T	T	T	T
F	F	F	T	T	T	T

Logical Implication (논리 함축)

- 수학적 논리에서의 논리 함축은 자연 언어에서의 의미와는 상당한 차이가 있다.
- 자연 언어에서는 "오늘 비가 오면, 야구 경기는 연기한다"라는 것은 가정과 결론 사이의 인과 관계를 나타내지만, 수학적 논리에서의 논리 함축 $P \rightarrow Q$ 는 P와 Q가 서로 연관이 없을지라도 임의의 두 명제를 결합할 수 있다.
- 왜냐하면 명제에서는 명제의 의미나 구조와 관계없이 오직 명제들의 진리값만을 다루기 때문이다.
- 논리 함축 $P \rightarrow Q$ 에 대해서 $Q \rightarrow P$ 를 역(converse)이라 한다.
- $\neg Q \rightarrow \neg P$ 를 대우(contrapositive)라 한다.
- $\neg P \rightarrow \neg Q$ 를 이(inverse)라고 한다.
- 즉, 역(converse)이란 명제의 가정과 결론에 대해서 가정은 결론으로 결론은 가정으로 위치를 바꾼 것을 말하고, 이(inverse)는 위치는 그대로 둔 채 가정과 결론들을 부정으로 만든 것을 말한다. 대우란 가정과 결론의 위치도 바꾸고 가정과 결론을 부정으로 바꾸는 것을 말한다.
- 명제가 참이면 대우도 반드시 참이 된다.

Understanding Implication₁

In $p \rightarrow q$ there does not need to be any connection between the antecedent or the consequent. The “meaning” of $p \rightarrow q$ depends only on the truth values of p and q .

These implications are perfectly fine, but would not be used in ordinary English.

- “If the moon is made of green cheese, then I have more money than Bill Gates. ”
- “If the moon is made of green cheese then I’m on welfare.”
- “If $1 + 1 = 3$, then your grandma wears combat boots.”

Different Ways of Expressing $p \rightarrow q$

if p , then q

p implies q

if p , q

p only if q

q unless $\neg p$

q when p

q if p

q whenever p

p is sufficient for q

q follows from p

q is necessary for p

a necessary condition for p is q

a sufficient condition for q is p

System Specifications

- System and Software engineers take requirements in English and express them in a precise specification language based on logic.
- **Example:** Express in propositional logic:
- “The automated reply cannot be sent when the file system is full”
- **Solution:** One possible solution: Let p denote “The automated reply can be sent” and q denote “The file system is full.”

$$q \rightarrow \neg p$$

Consistent System Specifications

- **Definition:** A list of propositions is *consistent* if it is possible to assign truth values to the proposition variables so that each proposition is true.
- **Exercise:** Are these specifications consistent?
 - “The diagnostic message is stored in the buffer or it is retransmitted.”
 - “The diagnostic message is stored in the buffer.”
 - “If the diagnostic message is stored in the buffer, then it is retransmitted.”
- **Solution:** Let p denote “The diagnostic message is stored in the buffer.” Let q denote “The diagnostic message is retransmitted” The specification can be written as: $p \vee q, \neg p, p \rightarrow q$. When p is false and q is true all three statements are true. So the specification is consistent.
 - What if “The diagnostic message is not retransmitted” is added.
Solution: Now we are adding $\neg q$ and there is no satisfying assignment. So the specification is not consistent.

APPLICATIONS OF PROPOSITIONAL LOGIC

Translating English Sentences

- Steps to convert an English sentence to a statement in propositional logic
 - Identify atomic propositions and represent using propositional variables.
 - Determine appropriate logical connectives
- “If I go to Harry’s or to the country, I will not go shopping.”
 - p : I go to Harry’s
 - q : I go to the country.
 - r : I will go shopping.

If p or q then not r .

$$(p \vee q) \rightarrow \neg r$$

Example

Problem: Translate the following sentence into propositional logic:

“You can access the Internet from campus only if you are a computer science major or you are not a freshman.”

One Solution: Let a , c , and f represent respectively “You can access the internet from campus,” “You are a computer science major,” and “You are a freshman.”

$$a \rightarrow (c \vee \neg f)$$

PROPOSITIONAL EQUIVALENCES

Logical Equivalences 논리적 동치

논리적 동치

- 임의의 명제 P와 Q에 대해서 논리적 동치 $P \Leftrightarrow Q$ 의 진리값은 P와 Q가 모두 참이거나 모두 거짓일 경우에만 참이 되고, 그 이외의 경우에는 모두 거짓이 된다.
- $P \Leftrightarrow Q$ 는 논리 함축 $P \rightarrow Q$ 와 논리 함축 $Q \rightarrow P$ 의 논리곱이다.

P	Q	$P \rightarrow Q$	$Q \rightarrow P$	$(P \rightarrow Q) \wedge (Q \rightarrow P)$	$P \Leftrightarrow Q$
T	T	T	T	T	T
T	F	F	T	F	F
F	T	T	F	F	F
F	F	T	T	T	T

[표 2-6] 논리적 동치를 나타내는 진리표

P	Q	$P \Leftrightarrow Q$
T	T	T
T	F	F
F	T	F
F	F	T

Logical Equivalences 논리적 동치

지금까지 설명한 합성 명제에 대한 진리표

[표 2-7] 논리 연산들에 대한 진리표

P	Q	$P \wedge Q$	$P \vee Q$	$\neg P$	$P \oplus Q$	$P \rightarrow Q$	$P \Leftrightarrow Q$
T	T	T	T	F	F	T	T
T	F	F	T	F	T	F	F
F	T	F	T	T	T	T	F
F	F	F	F	T	F	T	T

합성 명제 $((P \vee Q) \wedge \neg R) \rightarrow P$ 에 대한 진리표를 구하라.

P	Q	R	$P \vee Q$	$\neg R$	$(P \vee Q) \wedge \neg R$	$((P \vee Q) \wedge \neg R) \rightarrow P$
T	T	T	T	F	F	T
T	T	F	T	T	T	T
T	F	T	T	F	F	T
T	F	F	T	T	T	T
F	T	T	T	F	F	T
F	T	F	T	T	T	F
F	F	T	F	F	F	T
F	F	F	F	T	F	T



연산자 우선순위표

- 합성 명제에 대해서 진리표를 작성하기 위해서는 우선순위가 정해져야 한다.
- 우선순위를 위해서 괄호를 사용하기도 하지만, 이 방법은 합성 명제가 길고 복잡하다면 너무나 많은 괄호 때문에 가독성(readability)이 떨어진다.
- 가독성을 높이기 위해 연산자 우선순위표(operator precedence table) 사용
- 연산자들의 우선순위와 결합법칙을 이용하여 우선순위를 정한다.

논리 연산자들의 우선 순위

- $\neg$ (부정) > $\wedge$ (논리곱) > $\vee$ (논리합) > $\rightarrow$ (논리 함축) > $\leftrightarrow$ (논리적 동치)
- ✓ 이러한 연산자 우선순위를 유지하고 $\neg$ (부정)은 오른쪽 결합법칙을, 나머지 연산자들은 왼쪽 결합법칙을 갖는다고 하자.
- ✓ 그러면 합성 명제 $P \wedge Q \rightarrow \neg P \vee Q \wedge R$ 에 괄호를 이용해서 표시하면 $(P \wedge Q) \rightarrow ((\neg P) \vee (Q \wedge R))$ 다.

Operator precedence 연산자 우선순위표

연산자 우선순위표를 활용한 연산 결과

- 연산자 우선순위표를 만들기 위해서는 주어진 합성 명제에서 사용하는 연산자들을 정방 행렬로 나열한다.
- 연산자들은 합성 명제에 나와 있는 것만 사용한다고 가정하므로 합성 명제 $P \wedge Q \rightarrow \neg P \vee Q \wedge R$ 에 대해서는 연산자가 $\neg, \wedge, \rightarrow, \vee$ 가 사용된다. 그리고 두 개의 스택(연산자, 피연산자)을 사용한다고 가정한다.
- 연산자 우선순위표의 초기 상태는 다음과 같다.

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$				
$\wedge$				
$\rightarrow$				
$\vee$				

- 1) 연산자가 동일할 때
 - 오른쪽 결합 법칙 : 입력 연산자 우선
 - 왼쪽 결합 법칙 : 스택 연산자 우선
- 2) 연산자가 다를 때

Operator precedence 연산자 우선순위표

- 이 표에서 **열**에 있는 연산자들은 **스택** 속에 있는 연산자들이고, **행**에 있는 연산자들은 **입력**에 있는 연산자들이다.
- 모든 항목들에 대해서 우선순위를 부여해보자.
 - ✓ 우선 1행과 1열의 항목에 우선순위를 생각해보자. 스택 속에는 연산자 $\neg$ 가 있고, 입력에도 연산자가 $\neg$ 가 있는 경우이다. 이 경우에 누구 먼저 연산해야 할까?
 - ✓ 같은 연산자이므로 연산자 우선순위가 같다.
 - ✓ 따라서 결합법칙이 무엇인지 살펴봐야 한다. **오른쪽 결합 법칙**을 가지므로 **입력에 $\neg$ 연산을 먼저 수행**해야 한다.
 - ✓ 그러므로 1행 1열의 우선순위는 입력에 있는 연산자가 스택 속에 있는 연산자보다 우선순위가 높다.

[행] 입력 연산자

[열]
스택
연산자

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<			
$\wedge$				
$\rightarrow$				
$\vee$				

Operator precedence 연산자 우선순위표

- 같은 방법으로 주대각 원소에 있는 우선순위를 생각해보자.
 - ✓ 다른 연산자 우선순위가 같고 **왼쪽 결합법칙**을 취하므로 다음과 같이 **스택 속에 있는 연산자의 우선순위가 높다.**

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<			
$\wedge$		>		
$\rightarrow$			>	
$\vee$				>

- 나머지 항목들도 채워보자.
 - ✓ 연산자 $\neg$ 의 우선순위가 가장 높다고 했으므로 다음과 같이 연산자 $\neg$ 가 다른 연산자들보다 모두 우선순위가 높다.

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<	>	>	>
$\wedge$	<	>		
$\rightarrow$	<		>	
$\vee$	<			>

Operator precedence 연산자 우선순위표

- 두 번째로는 $\wedge$ (논리곱)이 우선순위가 높으므로 다음과 같이 항목들을 채울 수 있다.

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<	>	>	>
$\wedge$	<	>	>	>
$\rightarrow$	<	<	>	
$\vee$	<	<		>

- 세 번째로는 $\vee$ (논리합)이 우선순위가 높으므로 다음과 같이 항목들을 채울 수 있다.

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<	>	>	>
$\wedge$	<	>	>	>
$\rightarrow$	<	<	>	<
$\vee$	<	<	>	>

- 이제 연산자 우선순위표를 이용해서 괄호를 사용하는 경우와 같은 결과가 나오는지 두 개의 스택을 사용해서 연산해보자.

Operator precedence 연산자 우선순위표

예제 2-14 연산자 우선순위표를 이용한 연산

임의의 명제 P, Q, R 에 대해서 합성 명제 $P \wedge Q \rightarrow \neg P \vee Q \wedge R$ 의 연산자 우선순위가 $\neg$ (부정) $>$ $\wedge$ (논리곱) $>$ $\vee$ (논리합) $>$ $\rightarrow$ (논리 함축)이고, $\neg$ (부정)은 오른쪽 결합법칙을 갖고 다른 논리 연산자들은 왼쪽 결합법칙을 갖는다고 하자. 이때 연산자 우선순위표를 만들고 두 개의 스택을 이용해서 연산해보자.

풀이

앞에서 연산자 우선순위표를 다음과 같이 만들었다.

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	$<$	$>$	$>$	$>$
$\wedge$	$<$	$>$	$>$	$>$
$\rightarrow$	$<$	$<$	$>$	$<$
$\vee$	$<$	$<$	$>$	$>$

이 표를 이용하여 합성 명제 $P \wedge Q \rightarrow \neg P \vee Q \wedge R$ 에 대해서 계산해보자. 입력은 왼쪽에서 오른쪽으로 하나씩 읽는다.

Operator precedence

	$\neg$	$\wedge$	$\rightarrow$	$\vee$
$\neg$	<	>	>	>
$\wedge$	<	>	>	>
$\rightarrow$	<	<	>	<
$\vee$	<	<	>	>

입력	연산자 스택	피연산자 스택	비고
$P \wedge Q \rightarrow \neg P \vee Q \wedge R$			P를 읽는다.
$\wedge Q \rightarrow \neg P \vee Q \wedge R$		P	피연산자 스택에 삽입 ^{push}
$Q \rightarrow \neg P \vee Q \wedge R$	$\wedge$	P	연산자 스택에 삽입
$\rightarrow \neg P \vee Q \wedge R$	$\wedge$	PQ	피연산자 스택에 삽입
$\rightarrow \neg P \vee Q \wedge R$	$\wedge$	PQ	연산자 우선순위 비교, 삭제 ^{pop}
$\rightarrow \neg P \vee Q \wedge R$		$(P \wedge Q)$	$P \wedge Q$ 연산 후 결과 삽입
$\neg P \vee Q \wedge R$	$\rightarrow$	$(P \wedge Q)$	연산자 스택에 삽입
$P \vee Q \wedge R$	$\rightarrow \neg$	$(P \wedge Q)$	연산자 우선순위 비교, 삽입
$\vee Q \wedge R$	$\rightarrow \neg$	$(P \wedge Q)P$	피연산자 스택에 삽입
$\vee Q \wedge R$	$\rightarrow \neg$	$(P \wedge Q)P$	연산자 우선순위 비교, 삭제
$\vee Q \wedge R$	$\rightarrow$	$(P \wedge Q)(\neg P)$	연산자 우선순위 비교, 삽입
$Q \wedge R$	$\rightarrow \vee$	$(P \wedge Q)(\neg P)$	피연산자 스택에 삽입
$\wedge R$	$\rightarrow \vee$	$(P \wedge Q)(\neg P)Q$	연산자 스택에 삽입
R	$\rightarrow \vee \wedge$	$(P \wedge Q)(\neg P)Q$	피연산자 스택에 삽입
	$\rightarrow \vee \wedge$	$(P \wedge Q)(\neg P)QR$	삭제

Operator precedence 연산자 우선순위표

입력	연산자 스택	피연산자 스택	비고
	$\rightarrow \vee$	$(P \wedge Q)(\neg P)(Q \wedge R)$	삭제
	$\rightarrow$	$(P \wedge Q)((\neg P) \vee (Q \wedge R))$	삭제
		$(P \wedge Q) \rightarrow ((\neg P) \vee (Q \wedge R))$	

그러므로 다음과 같이 연산된다. 이 결과는 처음에 예측한 괄호 순서와 똑같다.

$$(P \wedge Q) \rightarrow ((\neg P) \vee (Q \wedge R))$$

일반적인 합성 명제에 대해 진리표를 구하는 프로그램을 작성하라. (2주)

❖ 조건

- 명제 변수는 많아야 3개로 제한하고, 논리 연산자도 많아야 4종류만 허용한다.
- 세 개의 명제 P, Q, R이 있고, 논리연산자는 and, or, not, implication 네 가지가 있다.
- 세 개의 명제와 네 개의 논리 연산자를 조합하여 합성 명제를 입력으로 주면 그에 대한 진리표가 출력되는 프로그램을 작성하라.
- 단, 현재 우리가 사용하고 있는 논리 연산자의 우선 순위와 결합 법칙대로 작성한다.
 - ✓ 논리 연산자들의 우선순위는 $\neg$ (부정) > $\wedge$ (논리곱) > $\vee$ (논리합) > $\rightarrow$ (논리 함축) 과 같고 $\neg$ (부정)은 오른쪽 결합법칙을, 나머지 연산자들은 왼쪽 결합법칙을 갖는다.
- 합성 명제 $P \wedge Q \rightarrow \neg P \vee Q \wedge R$ 에 대해 진리표를 만들라.

❖ 연산자 우선순위, 결합법칙을 이용한 우선 순위표 만들기

❖ 진리값, 진리값 출력(전체의 합성 명제에 대한 진리표가 출력으로)

❖ 2장 프로그래밍 실습 1번을 참조

명제 종류

명제의 종류 : 항진(tautology)명제, 모순(contradiction)명제, 사건(contingency)명제

정의 2-3 항진 명제

항진 명제^{tautology}는 항상 참의 진리값을 가지는 명제를 말한다.

정의 2-4 모순 명제

모순 명제^{contradiction}는 항상 거짓의 진리값을 가지는 명제를 말한다.

❖ 사건(contingency) 명제 : 항진 명제도 아니고 모순 명제도 아닌 명제

Identity

항등 (Identity) $\Leftrightarrow$

- 논리적 동치(equivalence)란 P와 Q가 똑같은 진리값을 갖는 것을 말하고 논리적으로 동치인 명제로 명제들을 간단히 할 때 사용한다.

[표 2-8] 항등

번호	항등 관계	설명
1	$P \Leftrightarrow (P \vee P)$	$\vee$ 의 멱등법칙
2	$P \Leftrightarrow (P \wedge P)$	$\wedge$ 의 멱등법칙
3	$(P \vee Q) \Leftrightarrow (Q \vee P)$	$\vee$ 의 교환법칙
4	$(P \wedge Q) \Leftrightarrow (Q \wedge P)$	$\wedge$ 의 교환법칙
5	$[(P \vee Q) \vee R] \Leftrightarrow [P \vee (Q \vee R)]$	$\vee$ 의 결합법칙
6	$[(P \wedge Q) \wedge R] \Leftrightarrow [P \wedge (Q \wedge R)]$	$\wedge$ 의 결합법칙
7	$\neg(P \vee Q) \Leftrightarrow (\neg P \wedge \neg Q)$	드 모르간 법칙
8	$\neg(P \wedge Q) \Leftrightarrow (\neg P \vee \neg Q)$	드 모르간 법칙
9	$[P \wedge (Q \vee R)] \Leftrightarrow [(P \wedge Q) \vee (P \wedge R)]$	$\wedge$ 의 분배법칙
10	$[P \vee (Q \wedge R)] \Leftrightarrow [(P \vee Q) \wedge (P \vee R)]$	$\vee$ 의 분배법칙
11	$(P \vee 1) \Leftrightarrow 1$	항등법칙
12	$(P \wedge 1) \Leftrightarrow P$	항등법칙
13	$(P \vee 0) \Leftrightarrow P$	항등법칙

[표 2-8] 항등

번호	항등 관계	설명
14	$(P \wedge 0) \Leftrightarrow 0$	항등법칙
15	$(P \vee \neg P) \Leftrightarrow 1$	여 법칙
16	$(P \wedge \neg P) \Leftrightarrow 0$	여 법칙
17	$P \Leftrightarrow \neg(\neg P)$	이중 부정
18	$(P \rightarrow Q) \Leftrightarrow (\neg P \vee Q)$	논리적 함축
19	$(P \Leftrightarrow Q) \Leftrightarrow [(P \rightarrow Q) \wedge (Q \rightarrow P)]$	논리적 동치
20	$[(P \wedge Q) \rightarrow R] \Leftrightarrow [P \rightarrow (Q \rightarrow R)]$	수출
21	$[(P \rightarrow Q) \wedge (P \rightarrow \neg Q)] \Leftrightarrow \neg P$	모순
22	$(P \rightarrow Q) \Leftrightarrow (\neg Q \rightarrow \neg P)$	대우

멱등법칙 (idempotent law) : 연산을 여러 번 적용하더라도 결과가 달라지지 않는 성질

Identity

예제 2-18 항등

다음 합성 명제를 항등을 이용해서 간단히 하라.

$$[(A \rightarrow B) \vee (A \rightarrow D)] \rightarrow (B \vee D)$$

풀이

[표 2-8]의 항등을 이용해서 간단히 하면 다음과 같다.

$$\begin{aligned} & [(A \rightarrow B) \vee (A \rightarrow D)] \rightarrow (B \vee D) \\ \Leftrightarrow & [(\neg A \vee B) \vee (\neg A \vee D)] \rightarrow (B \vee D) && (18\text{에 의해서}) \\ \Leftrightarrow & [(\neg A \vee \neg A) \vee (B \vee D)] \rightarrow (B \vee D) && (5\text{와 } 3\text{에 의해서}) \\ \Leftrightarrow & [\neg A \vee (B \vee D)] \rightarrow (B \vee D) && (1\text{에 의해서}) \\ \Leftrightarrow & \neg[\neg A \vee (B \vee D)] \vee (B \vee D) && (18\text{에 의해서}) \\ \Leftrightarrow & [\neg(\neg A) \wedge \neg(B \vee D)] \vee (B \vee D) && (7\text{에 의해서}) \\ \Leftrightarrow & [A \wedge \neg(B \vee D)] \vee (B \vee D) && (17\text{에 의해서}) \\ \Leftrightarrow & [A \vee (B \vee D)] \wedge [\neg(B \vee D) \vee (B \vee D)] && (10\text{에 의해서}) \\ \Leftrightarrow & (A \vee B \vee D) \wedge 1 && (15\text{에 의해서}) \\ \Leftrightarrow & A \vee B \vee D && (12\text{에 의해서}) \end{aligned}$$

PREDICATES AND QUANTIFIERS



명제의 표현 : 술어와 한정자

❖ 술어(Predicate)

정의 2-5 술어

술어^{predicate}란 한 객체의 **성질**이나 객체와 객체 사이의 **관계**를 표현하는 것을 말한다.

- “ x is tall.” 이라는 문장에서 x 를 변수라고 하고 tall을 **술어**라고 한다.
술어의 특별한 표현 방법으로 $T(x)$ 라고 표현할 수 있는데 여기서 T 를 술어라 하고 x 를 **개별 변수(individual variable)**라고 한다.
 - ✓ 개별 변수: 술어의 특별한 표현 방법에서 괄호 안에 나타나는 변수
- 일반적으로 P 를 술어라 하면 $P(x_1, x_2, \dots, x_n)$ 에서 $x_i[i = 1, \dots, n]$ 를 개별 변수라 하고, P 는 n 개의 인수(argument)를 갖는 술어라 한다.

[사전] Predicate : 문장 속에서 주어에 대해 진술하는 동사 이하 부분

❖ 명제와 술어와의 관계

- 술어는 명제로 바꿀 수 있다. 이렇게 하기 위해서는 술어의 개별 변수를 한정시켜 주어야 한다. 술어의 개별 변수를 한정시켜주는 방법에는 개별 변수를 한 개의 값으로 한정시켜 주는 방법과 개별 변수를 그 변수의 한정자에 의해서 한정하는 두 가지 방법이 있다.

예제 2-19 술어를 명제로 바꾸기

문장 " $x + y = 3$ "을 술어를 이용하여 표현하면 $P(x, y)$ 이다. 이를 명제로 바꿔라.

풀이

술어 : x, y 의 관계를 나타냄 ($P(x, y)$)

술어를 이용하여 표현한 $P(x, y)$ 는 명제가 아니다. 왜냐하면 참과 거짓 중에 하나를 가지지 못하기 때문이다. 이를 명제로 바꾸기 위해서는 술어 $P(x, y)$ 가 참이나 거짓 중에 하나를 가져야만 한다. 그러기 위해서는 개별 변수인 x 와 y 에 하나의 값을 대입해야 한다. 만약 $x = 1, y = 2$ 라 하면 술어 $P(1, 2)$ 는 참 명제가 된다.

Introducing Predicate Logic

Predicate logic uses the following new features:

- Variables: x, y, z
- Predicates: $P(x), M(x)$
- Quantifiers (*to be covered in a few slides*):

Propositional functions are a generalization of propositions.

- They contain variables and a predicate, e.g., $P(x)$
- Variables can be replaced by elements from their *domain*.

Propositional Functions

Propositional functions become propositions (and have truth values) when their variables are each replaced by a value from the *domain* (or *bound* by a quantifier, as we will see later).

The statement $P(x)$ is said to be the value of the propositional function P at x .

For example, let $P(x)$ denote " $x > 0$ " and the domain be the integers. Then:

$P(-3)$ is false.

$P(0)$ is false.

$P(3)$ is true.

Often the domain is denoted by U . So in this example U is the integers.



한정자(quantifier)

❖ 전체 한정자

- 기호 $\forall$ 로 표시하고 “**임의의**” 혹은 “**모든**” 등으로 읽는다. 만약 $P(x)$ 가 개별 변수 x 를 인수로 갖는 술어라고 하면 명제 “**임의의 x , $P(x)$** ”는 “ **x 의 모든 값들에 대해서 명제 $P(x)$ 는 참이다.**”로 해석한다.

❖ 존재 한정자

- 기호 $\exists$ 로 표시하고 “**어떤**” 혹은 “**적어도 하나에 대해서**”로 읽는다. 명제 “**어떤 x , $P(x)$** ”는 “**명제 $P(x)$ 가 참이 되는 x 의 값이 존재한다.**”로 해석한다.

❖ 유일 한정자

- 기호 $\exists!$ 로 표시하고 “ **$\sim$ 에 대한 유일한 x 가 존재한다.**”로 읽는다. 명제 “**어떤 x , $P(x)$** ”는 “**명제 $P(x)$ 가 참이 되는 x 의 값이 유일하게 존재한다**”로 해석한다.

Quantifiers



Charles
Peirce
(1839-1914)

We need *quantifiers* to express the meaning of English words including *all* and *some*.

- "All men are Mortal."
- "Some cats do not have fur."

The two most important quantifiers are:

- *Universal Quantifier*, "For all," symbol: $\forall$
- *Existential Quantifier*, "There exists," symbol: $\exists$

We write as in $\forall x P(x)$ and $\exists x P(x)$.

$\forall x P(x)$ asserts $P(x)$ is true for every x in the *domain*.

$\exists x P(x)$ asserts $P(x)$ is true for some x in the *domain*.

The quantifiers are said to bind the variable x in these expressions.



한정자(quantifier)

❖ 전체 한정자

- 기호 $\forall$ 로 표시하고 “임의의” 혹은 “모든”을 뜻한다. 명제 $P(x)$ 가 참이 되는 변수 x 를 인수로 갖는 술어라고 하며, “모든 값들에 대해서 명제 $P(x)$ 는 참이다”로 해석한다.

❖ 존재 한정자

- 기호 $\exists$ 로 표시하고 “어떤” 혹은 “일어”를 뜻한다. “어떤 x , $P(x)$ ”는 “명제 $P(x)$ 가 참이 되는 x 가 존재한다”로 해석한다.

❖ 유일 한정자

- 기호 $\exists!$ 로 표시하고 “~에 대한 유일한 x 가 존재한다.”로 읽는다. 명제 “어떤 x , $P(x)$ ”는 “명제 $P(x)$ 가 참이 되는 x 의 값이 유일하게 존재한다”로 해석한다.

C#, 접근 한정자 (Access quantifier)

모든 형식과 형식 멤버에는 액세스 수준이 있습니다. 액세스 수준은 사용 중인 어셈블리나 다른 어셈블리에서 형식 또는 형식 멤버를 사용할 수 있는지 여부를 제어합니다. 다음 액세스 한정자를 사용하여 형식 또는 멤버를 선언할 때 해당 항목의 액세스 수준을 지정할 수 있습니다.

- **public**: 동일한 어셈블리의 다른 코드나 해당 어셈블리를 참조하는 다른 어셈블리의 코드에서 형식이나 멤버에 액세스할 수 있습니다.
- **private**: 같은 `class` 또는 `struct`의 코드에서만 형식 또는 멤버에 액세스할 수 있습니다.
- **protected**: 같은 `class` 또는 해당 `class`에서 파생된 `class`의 코드에서만 형식 또는 멤버에 액세스할 수 있습니다.
- **internal**: 동일한 어셈블리의 코드에서는 형식이나 멤버에 액세스할 수 있지만 다른 어셈블리의 코드에서는 액세스할 수 없습니다.
- **protected internal**: 형식 또는 멤버가 선언된 어셈블리의 모든 코드에서 또는 다른 어셈블리의 파생 `class` 내에서 형식 또는 멤버에 액세스할 수 있습니다.
- **private protected**: 형식 또는 멤버가 선언된 어셈블리, 같은 `class`나 해당 `class`에서 파생된 형식의 코드에서만 형식 또는 멤버에 액세스할 수 있습니다.

[출처] Microsoft – C# 설명서

De Morgan's Laws for Quantifiers

- The rules for negating quantifiers are:

TABLE 2 De Morgan's Laws for Quantifiers.			
<i>Negation</i>	<i>Equivalent Statement</i>	<i>When Is Negation True?</i>	<i>When False?</i>
$\neg \exists x P(x)$	$\forall x \neg P(x)$	For every x , $P(x)$ is false.	There is x for which $P(x)$ is true.
$\neg \forall x P(x)$	$\exists x \neg P(x)$	There is an x for which $P(x)$ is false.	$P(x)$ is true for every x .

- The reasoning in the table shows that:

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

- These are important. You will use these.

Quantifier

예제 2-21 한정자와 술어로 명제 표현하기

한정자의 범위를 정수라 하고, $N(x)$ 는 음이 아닌 정수, $E(x)$ 는 짝수, $O(x)$ 는 홀수, $P(x)$ 는 소수라 하자. 다음의 명제들을 한정자와 술어를 이용해서 표현하라.

- (a) 짝수를 만족하는 x 가 존재한다.
- (b) 모든 정수는 짝수이거나 홀수이다.
- (c) 모든 소수들은 음이 아닌 정수이다.
- (d) 소수 중에서 유일한 짝수는 2이다.
- (e) 소수 중에서 짝수는 유일하다.

풀이

- (a) $\exists x, [E(x)]$
- (b) $\forall x, [E(x) \vee O(x)]$
- (c) $\forall x, [P(x) \rightarrow N(x)]$
- (d) $\forall x, [E(x) \wedge P(x) \rightarrow x = 2]$
- (e) $\exists! x, [E(x) \wedge P(x)]$

Qualifier

수식어(qualifier)

- 영문법의 형용사와 유사한 역할을 하는 단어를 사용하여 이름을 구별하는 방법. 즉, 이름에 정보를 덧붙여 같은 이름들을 다르게 구별하는 것이다.
- C 언어에서 구조체 변수를 사용할 때 멤버참조연산자(.)를 사용하여 멤버를 참조하여 사용한다.
- 완전 수식어[Complete qualifier], 부분 수식어[sub qualifier]
- A 레코드 안에 B와 C 레코드가 있고, B 레코드 안에는 D와 E 레코드가 있다. C 레코드 안에는 F가 있고 D 레코드 안에는 B 레코드가 E 레코드 안에는 C 레코드가 다음과 같다.

A		
B		C
D	E	F
B	C	

- F 레코드를 참조하기 위해서는 A, C, F와 같이 참조하는 완전 수식어가 있고, F와 같이 참조하는 부분 수식어가 있다.



정의 2-6 논리적 추론

어떤 명제가 참인 것을 근거로 하여 다른 명제가 참인 것을 유도해내는 것을 논리적 추론 logical inference이라 하며, 이때 근거가 되는 명제를 가정 hypothesis 또는 전제 premise라고 하고, 유도되는 명제를 결론 conclusion이라 한다.

예제 2-22 가정과 결론

다음 명제들로부터 가정과 결론을 구하라.

“나는 똥똥하거나 흘쭉하다. 나는 흘쭉하지는 않다. 그러면 나는 똥똥하다.”

풀이

가정은 “나는 똥똥하거나 흘쭉하다.”와 “나는 흘쭉하지는 않다.”이고, 결론은 “나는 똥똥하다.”이다.

The Argument

We can express the premises (above the line) and the conclusion (below the line) in predicate logic as an argument:

$$\frac{\forall x (Man(x) \rightarrow Mortal(x)) \quad Man(Socrates)}{\therefore Mortal(Socrates)}$$

We will see shortly that this is a valid argument.

Inference 논리적 추론

논리적 추론을 하기 위한 기본 법칙

❖ **대체(substitution)** : 한 명제에 있는 어떤 표현을 그것과 똑같은 의미를 지닌 다른 표현으로 나타내는 것

[표 2-9] 항진 명제의 형태를 이용하는 추론의 법칙

	추론의 법칙	항진 명제의 형태	이름
p이다 따라서 p 또는 Q이다	$\begin{array}{l} P \\ \therefore P \vee Q \end{array}$	$P \rightarrow (P \vee Q)$	선언적 부가 disjunctive addition
p이면서 q이다 따라서 p이다	$\begin{array}{l} P \wedge Q \\ \therefore P \end{array}$	$(P \wedge Q) \rightarrow P$	단순화 simplification
p이다 p이면 q이다 따라서 q이다	$\begin{array}{l} P \\ P \rightarrow Q \\ \therefore Q \end{array}$	$[P \wedge (P \rightarrow Q)] \rightarrow Q$	전건 긍정 modus ponens (긍정 논법)
q가 아니다 p이면 q이다 따라서 p이다	$\begin{array}{l} \neg Q \\ P \rightarrow Q \\ \therefore \neg P \end{array}$	$[\neg Q \wedge (P \rightarrow Q)] \rightarrow \neg P$	후건 부정 modus tollens (부정 논법)

논리적 추론

[표 2-9] 항진 명제의 형태를 이용하는 추론의 법칙

	추론의 법칙	항진 명제의 형태	이름
p 또는 q이다 p가 아니다 따라서 q이다	$\begin{array}{l} P \vee Q \\ \neg P \\ \therefore Q \end{array}$	$[(P \vee Q) \wedge \neg P] \rightarrow Q$	선언 삼단 논법 disjunctive syllogism
p이면 q이다 q이면 r이다 따라서 p이면 r이다	$\begin{array}{l} P \rightarrow Q \\ Q \rightarrow R \\ \therefore P \rightarrow R \end{array}$	$[(P \rightarrow Q) \wedge (Q \rightarrow R)] \rightarrow (P \rightarrow R)$	가언 삼단 논법 hypothetical syllogism
p이다 q이다 따라서 p이고 q이다	$\begin{array}{l} P \\ Q \\ \therefore P \wedge Q \end{array}$	$(P \wedge Q) \rightarrow P \wedge Q$	논리곱 conjunction
p이면 q이고, r이면 s 이다 p이거나 r이다. 따라서 q이거나 s이다	$\begin{array}{l} (P \rightarrow Q) \wedge (R \rightarrow S) \\ P \vee R \\ \therefore Q \vee S \end{array}$	$[(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R)] \rightarrow (Q \vee S)$	구성적 양도 논법 constructive dilemma
p이면 q이고, r이면 s이다 q가 아니거나, s가 아니다 따라서 p가 아니거나 r이 아니다	$\begin{array}{l} (P \rightarrow Q) \wedge (R \rightarrow S) \\ \neg Q \vee \neg S \\ \therefore \neg P \vee \neg R \end{array}$	$[(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (\neg Q \vee \neg S)] \rightarrow (\neg P \vee \neg R)$	파괴적 양도 논법 destructive dilemma

Rules of Inference for Propositional Logic: Modus Ponens

$p \rightarrow q$	Corresponding Tautology:
$\frac{p}{\therefore q}$	
	$(q \wedge (p \rightarrow q)) \rightarrow q$

Example:

Let p be “It is snowing.”

Let q be “I will study discrete math.”

“If it is snowing, then I will study discrete math.”

“It is snowing.”

“Therefore , I will study discrete math.”

Modus Tollens

$$\begin{array}{l} p \rightarrow q \\ \hline \neg p \\ \hline \therefore \neg q \end{array}$$

Corresponding Tautology:

$$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$$

Example:

Let p be “it is snowing.”

Let q be “I will study discrete math.”

“If it is snowing, then I will study discrete math.”

“I will not study discrete math.”

“Therefore , it is not snowing.”

Hypothetical Syllogism

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$$

Corresponding Tautology:

$$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$$

Example:

Let p be "it snows."

Let q be "I will study discrete math."

Let r be "I will get an A."

"If it snows, then I will study discrete math."

"If I study discrete math, I will get an A."

"Therefore , If it snows, I will get an A."

Simplification

Corresponding Tautology:

$$\frac{p \wedge q}{\therefore p}$$

$$(p \wedge q) \rightarrow p$$

Example:

Let p be "I will study discrete math."

Let q be "I will study English literature."

"I will study discrete math and English literature"

"Therefore, I will study discrete math."

Conjunction

$$\frac{p}{q} \\ \therefore p \wedge q$$

Corresponding Tautology:

$$((p) \wedge (q)) \rightarrow (p \wedge q)$$

Example:

Let p be "I will study discrete math."

Let q be "I will study English literature."

"I will study discrete math."

"I will study English literature."

"Therefore, I will study discrete math and I will study English literature."

Resolution

$$\neg p \vee r$$

$$\frac{p \vee q}{\therefore q \wedge r}$$

Resolution plays an important role in AI and is used in Prolog.

Corresponding Tautology:

$$((\neg p \vee r) \wedge (p \vee q)) \rightarrow (q \vee r)$$

Example:

Let p be "I will study discrete math."

Let r be "I will study English literature."

Let q be "I will study databases."

"I will not study discrete math or I will study English literature."

"I will study discrete math or I will study databases."

"Therefore, I will study databases or I will study English literature."

예제 2-23 항진 명제의 형태를 이용해 추론하기

다음과 같은 명제들을 생각해보자.

- [1] “만약 2가 홀수이거나 소크라테스가 사람이 아니라면 지구는 회전하지 않을 것이다. 만약 지구가 회전하지 않는다면 소금은 짜지 않을 것이다. [2] 그러나 소금은 짜다. 그러므로 소크라테스는 사람이다.”

결론

이 명제에서 앞의 3개 명제는 가정이고 마지막 명제는 결론이다. 여기서 가정이 참이라면 결론이 참인지 아닌지를 결정하라.

풀이 명제들을 다음과 같이 표시해보자.

F : 2는 짝수이다.

A : 소크라테스는 사람이다.

M : 지구는 회전한다.

P : 소금은 짜다.

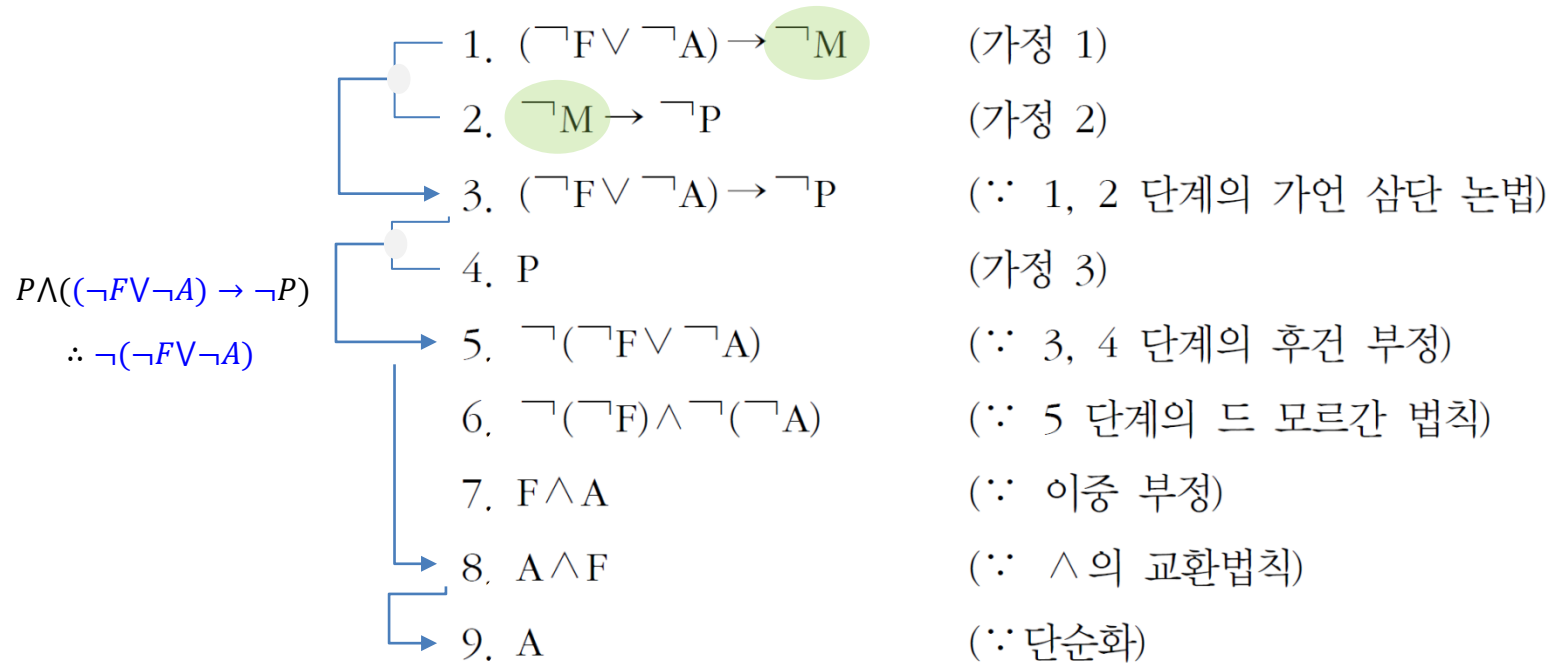
논리적 추론

그러면 주어진 명제들을 다음과 같이 나타낼 수 있다.

1. $(\neg F \vee \neg A) \rightarrow \neg M$
2. $\neg M \rightarrow \neg P$
3. P
4. A

가정 1, 2, 3이 참이라면 4가 참인지 아닌지를 결정하면 된다. [표 2-9]의 추론의 법칙들을 이용해서 결론이 참인지 아닌지를 결정하면 된다.

논리적 추론



그러므로 가정이 참이라고 하면 결론도 참이 된다.

$\neg Q$ $P \rightarrow Q$ $\therefore \neg P$	$[\neg Q \wedge (P \rightarrow Q)] \rightarrow \neg P$	후건 부정 modus tollens
--	--	---------------------

2.3 증명 기술과 프로그램 검증

Proof

정의 2-7 증명

증명^{proof}이란 가정에서 논리적 법칙을 이용하여 결론을 이끌어내는 것을 말한다. 이때 추론이 참이면 **진위**^{valid} 추론이라 하고, 추론이 거짓이면 **허위**^{fallacious} 추론이라 한다.

예제 2-24 진위 추론 혹은 허위 추론

다음과 같은 명제가 진위 추론임을 보여라.

“내가 공부를 한다면 나는 시험에 실패하지 않을 것이다. 내가 농구를 하지 않는다면 나는 공부를 할 것이다. 그러나 나는 시험에 실패했다. 그러므로 나는 농구를 했다.”

풀이

명제들을 다음과 같이 표시해보자.

Proof

P : 나는 공부한다.

Q : 나는 시험에 실패하지 않는다.

R : 나는 농구하지 않는다.

그러면 주어진 명제들을 다음과 같이 나타낼 수 있다.

1. $P \rightarrow Q$

2. $R \rightarrow P$

3. $\neg Q$

4. $\neg R$

가정 1, 2, 3이 참이라면 4가 참인지 아닌지를 결정하면 된다. [표 2-9]의 추론의 법칙을 이용해서 결론이 참인지 아닌지를 결정하자.

1. $P \rightarrow Q$ (가정 1)

2. $\neg Q$ (가정 3)

3. $\neg P$ ($\because$ 1, 2 단계의 후건 부정)

4. $R \rightarrow P$ (가정 2)

5. $\neg R$ ($\because$ 3, 4 단계의 후건 부정)

그러므로 가정이 참이라고 하면 결론도 참이 되므로 진위 추론이다.

Proof 방법

 **증명 문제** : 논리 함축 $P \rightarrow Q$ 를 증명하는 것이 대부분

❖ **증명 방법** : 직접 증명 방법, 간접 증명 방법

- 직접 증명 방법 : 진리를 근거로 논리 함축 $P \rightarrow Q$ 를 직접 증명하는 것
- 간접 증명 방법 : 논리적 동치를 이용하거나 다른 특수한 방법으로 증명하는 것
 - ✓ 가정된 명제로부터 직접 결론을 이끌어 내기가 쉽지 않은 경우에
간접적으로 결론이 참인 것을 이끌어 내는 방법
- 그 외 다음의 5가지로 나눌 수 있음

Proof 방법

논리 함축을 증명하기 위한 방법

(1) $P \rightarrow Q$ 의 **vacuous** 증명 방법

- P 가 거짓이라 하면 $P \rightarrow Q$ 는 항상 참이다.
그래서 $P \rightarrow Q$ 가 참인 것을 보이기 위해 P 가 거짓이라는 것만 보이면 된다.

(2) $P \rightarrow Q$ 의 **trivial** 증명 방법

- Q 가 참이라고 하면 $P \rightarrow Q$ 는 참이다.
그래서 $P \rightarrow Q$ 가 참인 것을 보이기 위해 Q 가 참이라는 것만 보이면 된다.

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

예제 2-25 trivial 증명 방법

U 를 전체 영역이라 하고 A 를 하나의 집합이라고 할 때, $A \subseteq U$ 임을 보여라.

증명

집합 A 가 U 의 부분집합이 되기 위한 필요충분조건은 $x \in U$ 에 속한 모든 x 에 대해서 $x \in A$ 임을 보여주는 것이다. 즉, $x \in A \rightarrow x \in U$ 가 참이 되는 것이다. 그러나 $x \in U$ 는 항상 참이므로 이 논리 함축은 $P \rightarrow Q$ 의 trivial 증명 방법에 의해서 참이다.

Proof 방법

(3) $P \rightarrow Q$ 의 직접 증명 방법

- P 가 참이라고 하면 $P \rightarrow Q$ 가 참이기 위해서는 Q 도 참이어야 한다.

예제 2-26 직접 증명 방법(1)

만약 $6x + 9y = 101$ 이면 x 또는 y 가 정수가 아님을 증명하라.

<u>증명</u>	$6x + 9y = 101$	$\rightarrow$	x 또는 y 가 정수가 아님
	P		Q

$P \rightarrow Q$ 의 직접 증명 방법에서 P 가 참이라고 가정하므로 $6x + 9y = 101$ 이 참이다. $6x + 9y = 101$ 은 $3(2x + 3y) = 101$ 이다. 그런데 101이 3으로 나누어떨어지지 않으므로 $2x + 3y$ 는 정수가 아니다. 그러므로 x 또는 y 는 정수가 아니다.

(4) 간접 증명(indirect proof) 방법

- 가정된 명제로부터 추론에 의해서 결론에 직접 도달되기가 어려운 경우에 간접적으로 결론이 참인 것을 이끌어내는 방법
- 대우(Contraposition)를 이용하는 방법, 배리법(proof the contradiction), 존재 증명(existence proof) 등 3가지로 나누어 볼 수 있다.



Proof 방법

- ① 대우(contraposition)를 이용하는 방법 : $P \rightarrow Q$ 와 $\neg Q \rightarrow \neg P$ 가 논리적으로 동치이므로 $\neg Q \rightarrow \neg P$ 가 참인 것을 증명함으로써 $P \rightarrow Q$ 가 참인 것을 보여주는 것

정의 2-8 완전수 [6, 28, 496, 8128, ...]

완전수 perfect number는 그의 약수 divisor들 중 자신을 제외한 모든 약수들의 합과 같은 수이다.

예제 2-28 대우를 이용하는 간접 증명 방법

“완전수는 소수가 아니다.”가 참인 것을 증명하라.

증명

대우를 이용해서 증명한다. 이 명제의 대우는 “소수는 완전수가 아니다.”이다. “완전수는 소수가 아니다.”가 참인 것을 증명하기 위해서 “소수는 완전수가 아니다.”가 참인 것을 보이면 된다.

q 를 소수라고 하면 $q \geq 2$ 이고 q 의 약수는 1과 q 이다. 이때 q 를 제외한 약수는 1밖에 없으므로 합은 1이다. 따라서 q 를 소수라고 가정하니 q 는 완전수가 아니다. 그러므로 완전수는 소수가 아니다.



② 배리법 : 명제 P 가 참인 것을 보여주는데 $\neg P$ 가 거짓임을 보이는 것

[귀류법 또는 반증법이라고도 함, "Proof by Contradiction" 모순에 의한 증명]

예제 2-29 배리법을 이용하는 간접 증명 방법

“가장 큰 소수 largest prime number는 존재하지 않는다.”가 참인 것을 증명하라.

풀이

전제 ($\sim P$) : 가장 큰 소수는 존재한다

가장 큰 소수 q 가 존재한다면 모든 소수는 1보다 크고 q 보다 크지 않으므로 소수의 개수는 유한하다. 2부터 q 까지의 소수를 곱한 것을 r 이라 하면 $r = 2 \times 3 \times 5 \times 7 \times \dots \times q$ 이다. 이때 $r+1$ 은 소수이다. 왜냐하면 $r+1$ 을 2부터 q 까지의 어느 소수로 나누어도 나머지가 1이기 때문이다. 이는 1과 $r+1$ 을 제외한 어떠한 두 정수의 곱으로도 $r+1$ 을 나타낼 수 없다. $r > q$ 이고 $r+1$ 은 q 보다 큰 소수이므로 q 가 가장 큰 소수라는 전제는 거짓이다. 그러므로 주어진 명제는 참이다.

Proof 방법

- ③ 존재 증명 방법 : $P(x)$ 를 x 라는 변수를 갖는 명제라고 할 때, $P(x)$ 가 참인 x 가 존재함을 보이는 것이다. 즉, $\exists x P(x)$.

예제 2-30 존재 증명 방법을 이용하는 간접 증명 방법

$a(a \neq 0)$, b 가 실수이면 방정식 $ax + b = 0$ 을 만족시키는 실수 x 가 존재함을 존재 증명 방법으로 보여라.

풀이

방정식 $ax + b = 0$ 은 a 가 0이 아니므로 양변을 a 로 나누어 이항하면 $x = -(b/a)$ 이다. 즉, 방정식을 만족하는 실수가 존재한다.

(5) 어떤 명제가 거짓임을 밝히기 위해 반증을 이용하는 방법

- ① 반례에 의한 반증의 방법 : $\forall x P[x]$ 의 형태를 가진 명제가 거짓임을 증명하는데 편리한 방법으로서 명제의 부정 $\exists x \neg P[x]$ 가 참임을 보이면 된다. 이때 x 를 반례라 한다.

예제 2-31 반례에 의한 반증의 방법

명제 “임의의 실수 x 에 대해 $x^2 > 0$ 이다.”를 반증하라.

증명

반례로서 0은 실수이며 $0^2 = 0$ 이다. $0 \ngtr 0$ 이므로 주어진 명제는 거짓이다.

contradiction

- ② 모순에 의한 반증의 방법 : 주어진 명제가 거짓임을 보이기 위해서 그 명제가 참이라고 가정하여 이미 알고 있는 어떤 사실에 모순되는 결론을 유도하면 된다.

예제 2-32 모순에 의한 반증의 방법

명제 “홀수의 제곱은 짝수이다.”를 반증하라.

증명

홀수는 어떤 정수 i 에 대해 $2i+1$ 로 나타낼 수 있고, 짝수는 어떤 정수 j 에 대해 $2j$ 로 나타낼 수 있다. 주어진 명제에 의하면 $(2i+1)^2 = 2j$ 이다. 이는 $4i^2 + 4i + 1 = 2j$ 이다. 좌변은 2로 나누어떨어지지 않는데, 우변은 2로 나누어떨어지므로 모순이다. 그러므로 주어진 명제는 거짓이다.

Mathematical Induction

수학적 귀납법

❖ 새로운 결과를 얻기 위한 대표적인 논증 방법 : 연역법, 귀납법

• 연역법(deduction)

- ✓ 일반적인 사실(fact)이나 원리들을 전제로 하여 추론(inference)을 통하여 구체적인 새로운 사실을 이끌어내는 방법이다.
- ✓ 이 과정에서는 직접 보고 듣고 느끼는 등의 경험을 필요로 하지 않는 순수한 생각만 사용한다.
- ✓ 연역법의 가장 전형적인 예로 가언 삼단논법이 있다.
- ✓ “소크라테스는 사람이다”, “사람은 죽는다”, 그래서 “소크라테스는 죽는다” 이다.
즉, 가정으로부터 결론을 이끌어 내는 추론을 했다.

p이면 q이다
q이면 r이다
따라서 p이면 r이다

$P \rightarrow Q$
 $Q \rightarrow R$
 $\therefore P \rightarrow R$

$[(P \rightarrow Q) \wedge (Q \rightarrow R)] \rightarrow (P \rightarrow R)$

가언 삼단 논법 hypothetical syllogism

Mathematical Induction



수학적 귀납법

- 귀납법(induction)

- ✓ 연역법과는 정반대의 방식이다.
- ✓ 개별적인 사실들을 종합하여 일반적인 원리를 도출해내는 것이다.
- ✓ 관찰과 실험에 기반을 둔 가설을 귀납 추론을 통하여 일반적인 규칙을 입증하는 것이다.

- 수학적 귀납법(mathematical induction)

- ✓ 명제가 참이라는 것을 증명하기 위한 방법이다.
- ✓ 객체(object)들의 연속(sequence)을 정의하기 위해서 사용된다.

정의 2-9 수학적 귀납법

수학적 귀납법 mathematical induction은 k 를 어떤 정수라 할 때 어떤 명제 $P(n)$ 이 $n \geq k$ (단, $n \in \mathbb{Z}$)인 모든 수에 대해서 모두 참 true인 것을 보여주기 위한 방법으로, 다음과 같이 증명하는 방법을 말한다.

(1) $P(k)$ 는 참이다.

(2) $n \geq k$ 인 모든 정수 n 에 대해 $P(n)$ 가 참이라고 가정하면 $P(n+1)$ 도 참이다.



Mathematical Induction

예제 2-33 수학적 귀납법(1)

수학적 귀납법에 의하여 다음이 성립함을 보여라.

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2} \quad (\text{단, } n \geq 1)$$

증명

$P(n) : 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ 이라 하자.

(1) 기초 단계 : $P(1)$ 이 참인 것을 보여준다.

$$\text{좌변} = 1, \quad \text{우변} = \frac{1(1+1)}{2} = 1$$

그러므로 $P(1)$ 은 참이다.

(2) 귀납적 단계 : $P(n)$ 이 $n \geq 1$ 에 대해서 참이라는 것을 보여주기 위해 $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ 이라고 할 때 $P(n+1)$ 이 참이라는 것을 보여준다.

$$\begin{aligned} 1 + 2 + \cdots + n + (n+1) &= [1 + 2 + \cdots + n] + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) \quad (\because \text{가정에서 성립}) \\ &= \frac{n(n+1) + 2(n+1)}{2} \\ &= \frac{(n+1)(n+2)}{2} \end{aligned}$$

이로부터 $P(n+1)$ 이 참이라는 것을 알 수 있다. 즉, $P(n)$ 이 참이면 $P(n+1)$ 도 참이 된다.

따라서 수학적 귀납법에 의해 $n \geq 1$ 인 모든 정수에 대해서 $P(n)$ 은 참이다.



Program Verification

프로그램 검증(program verification) : 매우 어려운 문제

- 우리가 어떤 문제를 해결하기 위해 프로그램을 작성한다.
- 이 프로그램이 항상 정확한 결과를 생성한다는 것을 어떻게 확신할 수 있는가?
- 프로그래머에게는 너무나도 어려운 문제이다.
- 하지만 프로그래머 입장에서는 최대한 검증을 해야만 한다.
- 하지만 원칙적으로 모든 프로그램의 정확성을 검증하는 방법은 존재하지 않는다. [이것은 튜링(Alan Turing)에 의해 증명되었다.]

Program Verification

프로그램 검증(program verification) : 매우 어려운 문제

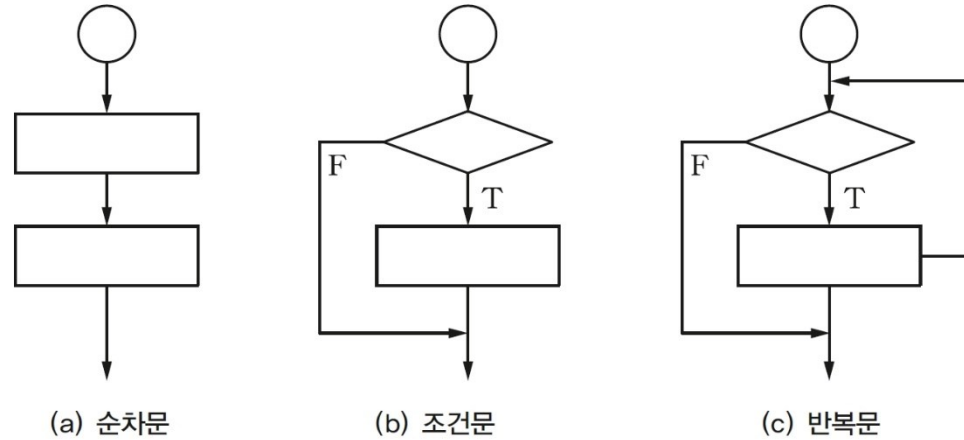
- 프로그램 검증은 소프트웨어 개발과 품질에 관한 부분이다.
- 소프트웨어의 품질이란 프로그램이 정상적으로 작동해야 하고, 고객이 의도한대로 요구사항을 올바르게 정의하고, 그 명시된 요구사항을 만족시키는 것을 말한다.
- 정확한 프로그램이 되기 위해서는 구문 에러(syntax error)를 포함하지 않아야 하고, 예상치 못하게 끝나서도 안 되며, 주어진 입력에 대해 정확한 결과를 도출해야만 한다.
- 즉, 수학적 검증 방법들을 통해 프로그램의 결과가 정확한지, 프로그램이 정상 종료되는지, 변수들의 값이 어떻게 변하는지 등을 검증하는 것이다.

프로그램의 검증 방법 : 정적(static)인 방법

- 프로그램을 실행하지 않고 결함을 찾아내는 것으로
여러 참여자들이 모여 프로그램을 검토하여 결함을 찾아내는 것이다.
- 수학적 귀납법을 포함하여 2장에서 설명한 추론 규칙과 증명 방법들을 사용하기도 하며,
프로그램을 일련의 부프로그램들로 분할하고 각 부프로그램들이 정확하다는 것을 보임으
로써 프로그램의 정확성을 증명하는 유용한 추론 규칙들도 있다.
- 정적으로 에러를 검출하는 프로그램 검증 도구 중 대표적인 것인 **컴파일러**이다.

정적인 방법

- 프로그램을 정적으로 검증하는 가장 기본적인 방법 : 각각의 문장들에 대한 기본 흐름도가 잘되어 있는지 확인하는 것



[그림 2-2] 순서도 표현

- 이 중에서 반복문에서 루프를 빠져나가는 문장과 goto와 같은 무조건 분기문을 사용할 경우에는 프로그램의 정확성을 검증하는 데 각별히 유의해야 한다.

- 소스 프로그램을 검토할 때 검토할 체크 리스트

- ✓ 요구사항 명세서에 기술된 기능은 충분히 코드에서 실행되는지?
- ✓ 요구사항 명세서에 기술되지 않은 추가적인 기능이 코드에서 실행되지는 않는지?
- ✓ 메소드(method) 리턴값은 적절하게 사용되는가?
- ✓ 모든 지역/전역 변수가 사용 전에 초기화되는가?
- ✓ 메소드는 정확하게 호출되는가?

소프트웨어 품질향상 위한 소스코드 평가 체크리스트

가. Clean Code 기반 소스코드 평가 체크리스트

구분	평가 항목	기준
형식	팀 규칙	- 한 팀 작성 코드는 일관성
	신문기사 방식	- 간단하면서도 설명적
	세로 형식	- 밀접한 개념은 세로 가까이
	가로 형식	- 소스 파일은 윤곽도 계층
이름	분명한 작성 의도	- 주석이 필요 없는 코드
	그릇된 정보 회피	- 길고 흡사한 이름 회피
	문맥에 맞는 단어	- 용도와 관계 있는 이름 사용
	클래스명 일관성	- 명사 또는 명사구
	함수명 일관성	- 동사 또는 동사구
함수	작은 단위	- 한 줄 블록 내 타 함수 호출
	한가지 기능	- 동일 레벨로 추상화
	작은 인수	- 최소한의 인수 사용
	중복 방지	- 동일 용도 회피
주석	코드 의도 표현	- 코드만으로 의도 표현 불가 시
	명쾌한 의도	- 제 3자의 관점으로 작성
	주석 삭제	- 없어도 되는 주석
오류	예외 사용	- Try-Catch 문 이용
	중복 방지	- 예외처리 중복 회피
	Null 반환금지	- Null 반환 경우 최소화

나. 시큐어코딩 기반 취약점 점검 소스코드 평가 체크리스트

구분	평가 항목	기준
입력데이터 검증 표현	디렉토리 경로	- 입력값 디렉토리 필터링
	스크립트 금지	- 업로드 시 특정 확장자 제한
	SQL Injection	- 입력 값 필터링
	XSS 공격 점검	- 게시물에 스크립트 제한
	서버 통제	- HTML, 웹APP 소스 리뷰
보안기능 포함여부	접근 제어	- 접근 권한 필요 부분 확인
	중요자원 권한	- 중요 자원 권한 부여 금지
	암호알고리즘	- 최신 암호알고리즘 사용
	중요정보 전송	- 중요정보 전송 시 암호화
	패스워드 저장	- 해시암호 등 일방 암호화
	취약 패스워드	- 비밀번호 조합 규칙 수립
에러처리	메시지 노출	- 오류메시지 코드 외부 노출
	오류상황 대응	- 오류 상황 대응 방안 수립
	적절한예외처리	- 의도하지 않는 상황 차단
코드오류	Null 포인터	- Null 설정된 변수 주소 참조
	부적절자원해제	- 메모리 자원 등 누수 방지

IEEE software life cycle

SQA – Software quality assurance IEEE 730

SCM – Software configuration management IEEE 828

STD – Software test documentation IEEE 829

SRS – **Software requirements specification IEEE 830**

V&V – Software verification and validation IEEE 1012

SDD – Software design description IEEE 1016

SPM – Software project management IEEE 1058

SUD – Software user documentation IEEE 1063

V • T • E

1. SRS의 특성(Nature of the SRS)

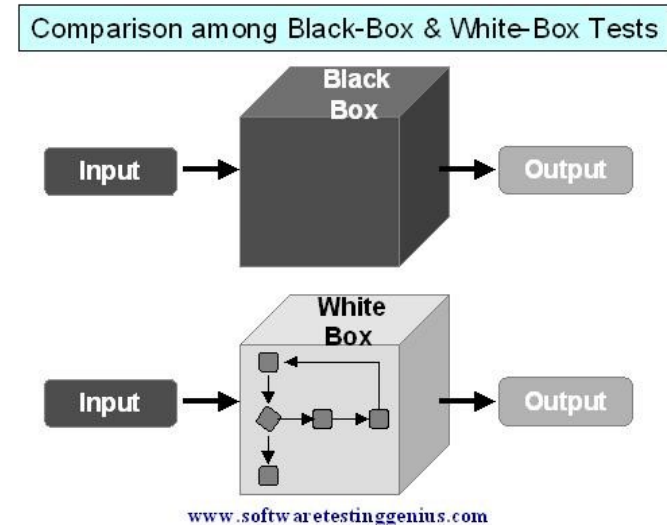
- SRS 작성자가 다루어야 할 기본 내용
 - i. 기능성(Functionality)
 - 소프트웨어가 수행하는 작업이 무엇인가?
 - ii. 외부 인터페이스(External interfaces)
 - 소프트웨어가 사람, 하드웨어, 다른 소프트웨어와 어떻게 상호작용 하는가?
 - iii. 성능(Performance)
 - 속도, 가용성, 반응 시간, 다양한 소프트웨어 기능의 복구 속도가 어느 정도인가?
 - iv. 속성(Attributes)
 - 이식성, 정확성, 유지 보수성, 보안성 및 고려사항이 무엇인가?
 - v. 구현에 부과된 설계 제약사항(Design constraints imposed on an implementation)
 - 적용해야 하는 표준, 구현하는 언어, 데이터베이스 무결성 정책, 리소스 제약사항, 운영체제가 무엇인가?

- 3. 상세 요구사항(Specific requirements)
 - 3.1. 외부 인터페이스 요구사항(External interface requirements)
 - 3.1.1. 사용자 인터페이스(User interfaces)
 - 3.1.2. 하드웨어 인터페이스(Hardware interfaces)
 - 3.1.3. 소프트웨어 인터페이스(Software interfaces)
 - 3.1.4. 통신 인터페이스(Communications interfaces)
 - 3.2. 기능 요구사항(Functional requirements)
 - 3.3. 성능 요구사항(Performance requirements)
 - 3.4. 논리적 데이터베이스 요구사항(Logical database requirements)
 - 3.5. 설계 제약사항(Design constraints)
 - 3.5.1. 표준 준수(Standards compliance)
 - 3.6. 소프트웨어 시스템 속성(Software system attributes)
 - 3.6.1. 신뢰성(Reliability)
 - 3.6.2. 가용성(Availability)
 - 3.6.3. 보안성(Security)
 - 3.6.4. 유지 보수성(Maintainability)
 - 3.6.5. 이식성(Portability)
 - 3.7. 상세 요구사항의 구성(Organizing the specific requirements)
 - 3.7.1. 시스템 모드(System mode)
 - 3.7.2. 사용자 부류(User class)
 - 3.7.3. 객체(Objects)
 - 3.7.4. 특징(Feature)
 - 3.7.5. 자극(Stimulus)
 - 3.7.6. 반응(Response)
 - 3.7.7. 기능 계층(Functional hierarchy)
 - 3.7.8. Additional comments

동적인 방법

🖋️ 프로그램의 검증 방법 : 동적인 방법

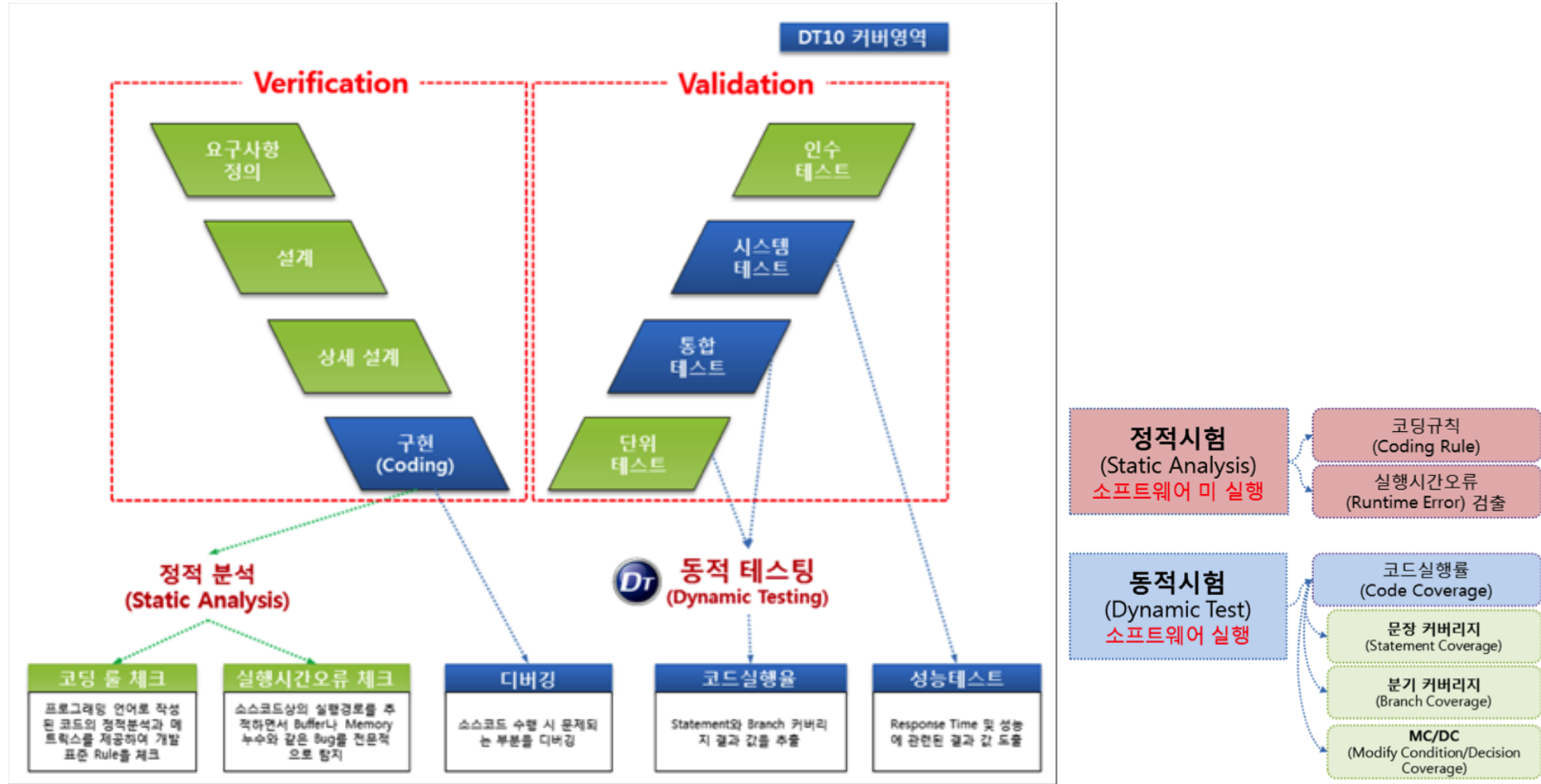
- 소프트웨어를 실행하여 결함을 찾아내는 방법으로 시험 데이터를 통하여 테스트하는 방법으로 결함을 찾아내는 방법이다.
- **Black-box testing**
 - ✓ 요구사항 명세 요구사항 명세서나 설계서를 참조하면서 수행
 - ✓ 로직에는 관심이 없고, 단지 입, 출력값을 테스트한다.
 - ✓ 주로 경계값들을 테스트한다.
- **White-box testing**
 - ✓ 소스 코드를 직접 참조하면서 수행한다.
 - ✓ 모든 코드가 한번은 실행되게 입력하는 문장 (statement)의 범위(coverage)와 분기(branch), 조건(condition), 다중 조건(multiple condition) 등을 주로 테스트 한다.



동적인 방법

Note

소프트웨어 테스트



정적,동적 방법 비교

정적 테스트 유형

유형	세부 기법	설명
흐름 분석	경로 분석	- 흐름 그래프 상 모든 경로 수행 - 변수들 사이 인과관계 확인
	제어 흐름 분석	- 모든 구분, 블록 조합 검사 - 기본, 문장, 분기, 조건 커버리지
	데이터 흐름 분석	- 프로그램 변수 정의/사용 시험 - All path, defs, uses 등
정형 분석 기법	정형 분석	- S/W 수학적 표현, 정확도 제고 - 안정성/보안성 중요 SW 적용
문법 체크	패턴 매칭	- 코드 표준 준수 여부 확인 - 텍스트 형태, 트리구조 패턴 매칭
보안 분석	보안 분석	- 보안 취약점 원인 위험도 분류 - 수정 권장사항 제공
분석 구간	지역 분석	- 하나의 함수만 대상
	모듈 레벨	- 클래스나 유닛 기준 분석
	전역 분석	- 전체 프로그램 대상 분석

모든 경로에 대해 분석하므로 TC(Test case)에 영향 없고,
Case 작성 소요 시간과 비용 절감

출처 : <http://blog.skbynet/소프트웨어-정적-테스트-동적-테스트/>

동적 테스트 유형

유형	세부 기법	설명
분석기	코드 커버리지	- 코드 함수 내 경로 분석 - 코드 커버리지 측정, McCabe IQ
	네트워크 분석기	- 네트워크 충돌 횟수 길이 초과 패킷 개수 정보 수집
	Assertion 분석기	- 프로그램 행위에 대한 Assertion 충족 여부 검사
체커	런타임 에러 체커	- 자동화 도구 이용 실행 시 - RTE Basis, Polyspace
	보안 체커	- 보안 취약점 검출 - 모의 해킹, 크래킹 시도
	상태 체커	- 메모리 할당, 누수 - 모듈 별 수행시간 분석
	메모리참조 에러 체커	- 코드 메모리 및 성능 프로파일링 - 빌드/컴파일 과정 소스 분석
분석 도구 및 기법	캡처-재생	- 실제 코드 동작 시 각 라인 추적
	동작 추적	- 함수 별 소모 시간 측정
	디버거	- JTAG, Trace32, APM Multi-ICE 등
	인스트루먼팅	- 소스코드에 메모리 동작 및 성능 분석 위한 별도 코드 삽입

동적 분석은 실제 동작 시 수행되므로 동적 분석 도구 연결
인터페이스 존재 필요(파일, 시리얼 등)



2.4 응용 : 지식 베이스 시스템

Knowledge-based System

지식 베이스 시스템(knowledge-based system)

- 전문가 시스템의 구성 요소의 하나로서, *인공지능 에이전트가 사용될 분야와 관련된 지적 활동과 경험을 통해서 축적한 전문 지식 그리고 문제 해결에 필요한 사실과 규칙 등이 저장되어 있는 데이터베이스 시스템*이다.
- 이 분야는 논리 추론 개념을 응용할 수 있는 컴퓨터 분야이다.

인공지능(artificial intelligence)

- 2016년 이세돌과 알파고(AlphaGo) : 인공지능 개념 관심
- 인공지능이란 인공적으로 만든 지능으로 지능이란 본능적이나 자동적으로 행동하는 대신에 생각하고 이해하여 행동하는 능력이다.
- 인공지능이라는 용어는 존 매카시(John McCarthy)가 제안하였고 사람의 생각과 관련된 활동으로 의사 결정, 문제 해결, 학습 등의 활동을 자동화하는 것을 말한다. 즉, 인간처럼 사고하고 감지하고 행동하도록 설계된 기계이다.
- 인공지능 분야에 있는 로봇틱스(robotics), 게임 놀이(game playing), 문제 해결(problem solving), 자연어 처리(natural language processing) 및 패턴 인식(pattern matching) 부분들에 대한 연구가 많이 진전했다.

[Wikipedia] - 에이전트

에이전트는 독자적으로 존재하지 않고 어떤 환경의 일부이거나 그 안에서 특정한 목적을 위해 사용자를 대신하여 동작하는 자율적 프로세스[시스템]이다. 여기서의 환경은 운영 체제, 네트워크 등을 지칭한다.

에이전트는 지식베이스와 추론 기능을 가지며 사용자, 자원(resource), 또는 다른 에이전트와의 정보교환과 통신을 통해 문제해결을 도모한다.

성질 : Autonomy[자율성], Social ability[사회성], Reactivity[반응성], Proactivity[능동성], Temporal continuity[시간 연속성], Goal-orientedness[목표지향성]

속성 : Mobility, Rationality, Adaptability, Collaboration

분류 : Learning agent, Interface agent, Desktop agent, Internet agent, Electronic commerce agent

Knowledge-based System

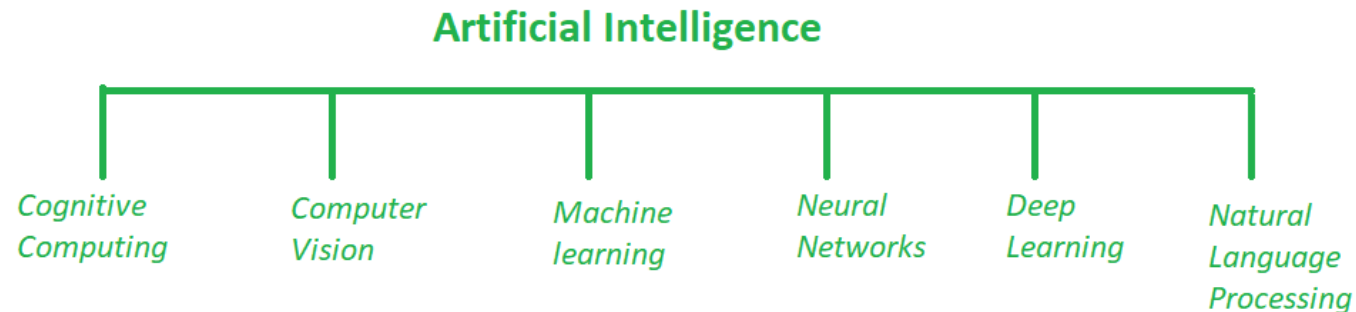
❖ 최근의 인공지능 : 기계학습과 딥 러닝이 대세

• Machine learning

- ✓ 인공지능의 한 분야로 알고리즘을 이용해 데이터를 분석하고, 분석을 통해 학습하며, 학습한 내용을 기반으로 판단이나 예측을 하는 분야이다.

• Deep learning

- ✓ 기계 학습의 한 분야로 뇌의 뉴런과 유사한 정보 입출력 계층을 활용해 데이터 학습하는 분야이다.
- ✓ 병렬 연산에 최적화된 GPU(Graphics Processing Unit)의 등장으로 진정한 딥 러닝이 등장했다.



[출처] <https://www.geeksforgeeks.org/difference-between-ai-and-expert-system/>

Knowledge-based System

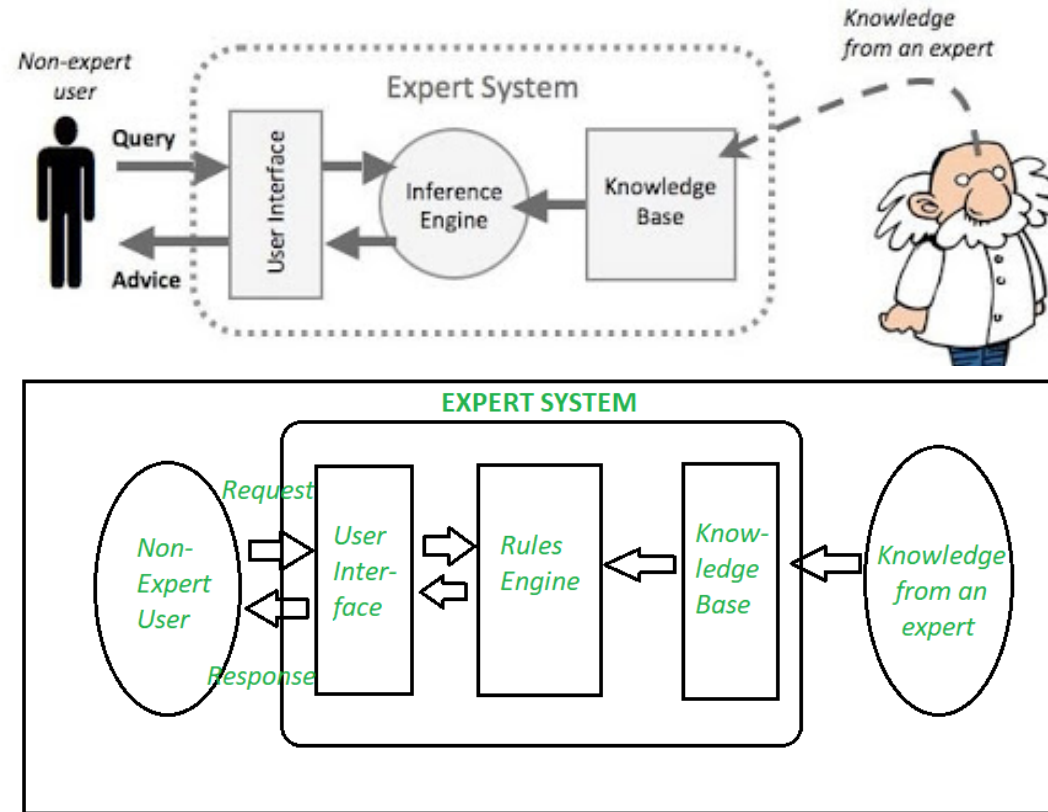
❖ Expert System

- 인공지능의 일부를 정규 논리(formal logic) 개념을 응용하여 기계가 전문가의 전문 지식을 추론을 하여 전문가와 동등한 수준에서 문제를 해결할 수 있도록 하는 것이 전문가 시스템(expert system)이다.
- 의료 진단(medical diagnosis), 논리 분석(logic analysis), 음성 이해(speech understanding), 화학, 수학 등의 분야에서 주요하게 수행되어 왔다.
- 전문가가 갖고 있는 전문 지식을 이용하고자 하는 시도를 처음 성공한 사람은 미국의 스탠퍼드 대학의 파이겐바움(Edward A. Feigenbaum) 교수이다.
- 가장 성공적인 작품은 MYCIN이다.
 - ✓ 의사가 환자를 진찰하여, 그 결과와 검사 결과의 세균명 등을 입력시키면 병명을 결정하여 어떤 항생 물질을 얼마 정도 복용시키면 좋을지 등과 같은 치료법을 알려주는 전문가 시스템이다.

Knowledge-based System

❖ Expert System

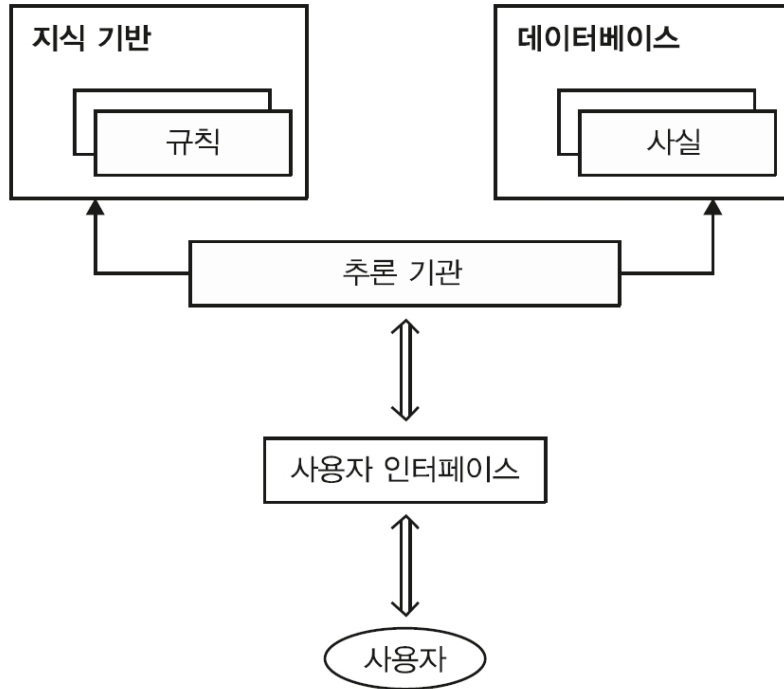
- 전문가 시스템은 **사실(fact)**들을 저장하는 데이터베이스, **추론 규칙(rule)**을 가지고 있는 지식 기반 시스템 그리고 사실들과 규칙으로부터 새로운 사실을 추론하는 **추론 기관(inference engine)**, 사용자들의 요구 사항을 받는 **사용자 인터페이스** 등으로 구성되어 있다.
- 추론 기관은 전문가 시스템에게 문제를 풀 수 있는 능력을 부여한다.



[출처] <https://sites.google.com/site/mrstevensonstechclassroom/hl-topics-only/4a-robotics-ai/5-expert-systems>

[출처] <https://www.geeksforgeeks.org/difference-between-ai-and-expert-system/>

Knowledge-based System



[그림 2-3] 전문가 시스템의 구조

- 사용자가 추론 기관에 사용자 요구 사항을 전달하면
- 주어진 문제의 해결을 위해 추론 기관은 지식 기반 시스템에 있는 규칙들과 사실들을 저장하고 있는 데이터베이스를 탐색하고 새로운 사실들을 추론하고 제어한다.
- 이렇게 해서 얻어진 새로운 사실들을 사용자에게 다시 전달해주는 시스템이다.

Knowledge-based System

ARTIFICIAL INTELLIGENCE	EXPERT SYSTEM
AI is the ability of a machine or a computer program to think, work, learn and react like humans.	Expert systems represent the most successful demonstration of the capabilities of AI.
AI involves the use of methods based on the intelligent behavior of humans to solve complex problems.	Experts systems are computer programs designed to solve complex decision problems.
Components of AI: 1. Natural Language Processing (NLP) 2. Knowledge representation 3. Reasoning 4. Problem solving 5. Machine learning	Components of expert system: 1. Inference engine 2. Knowledge base 3. User interface 4. Knowledge acquisition module
AI is the study is systems that act in a way to any observer would appear to be intelligent.	Expert system represent the most successful demonstration of the capabilities of AI
AI systems are used in a wide range of industries, from healthcare to finance, automotive, data security, etc.	Expert systems provide expert advice and guidance in a wide variety of activities.



Appendix

SymPy



SymPy 1.11
documentation

🔍 Search

Installation

Tutorials

How-to Guides

Explanations

API Reference

Basics

Code Generation

Logic

Logic

Sets

Matrices

Number Theory

Physics

Utilities

Topics

Contributing

Documentation Version

SymPy 1.11 (latest version)

Logic

Introduction

The logic module for SymPy allows to form and manipulate logic expressions using symbolic and Boolean values.

Forming logical expressions

You can build Boolean expressions with the standard python operators `&` (And), `|` (Or), `~` (Not):

```
>>> from sympy import *
>>> x, y = symbols('x,y')
>>> y | (x & y)
y | (x & y)
>>> x | y
x | y
>>> ~x
~x
```

You can also form implications with `>>` and `<<`:

```
>>> x >> y
Implies(x, y)
>>> x << y
Implies(y, x)
```

Like most types in SymPy, Boolean expressions inherit from `Basic`:

```
>>> (y & x).subs({x: True, y: True})
True
>>> (x | y).atoms()
{x, y}
```

The logic module also includes the following functions to derive boolean expressions from their truth tables:

`sympy.logic.boolalg.SOPform(variables, minterms, dontcares=None)` [\[source\]](#)

The SOPform function uses `simplified_pairs` and a redundant group- eliminating algorithm to convert the list of all input combos that generate '1' (the minterms) into the smallest sum-of-

Appendix

■ CLIPS

Facts and rules [\[edit\]](#)

CLIPS uses [forward chaining](#).^{[4][2]} Like other expert system languages, CLIPS deals with rules and facts. Various facts can make a rule applicable. An applicable rule is then fired.^[4] Facts and rules are created by first defining them, as shown below:

```
(deftemplate car_problem
  (slot name)
  (slot status))

(deffacts trouble_shooting
  (car_problem (name ignition_key) (status on))
  (car_problem (name engine) (status wont_start))
  (car_problem (name headlights) (status work)))

(defrule rule1
  (car_problem (name ignition_key) (status on))
  (car_problem (name engine) (status wont_start))
  =>
  (assert (car_problem (name starter) (status faulty))))
```